

CLSVH-ZT

Cross-Layer Security Validation Header: Research Summary and Implementation Roadmap

Executive summary

CLSVH-ZT should be built as a companion security framework. It does not replace IP, TCP, UDP, TLS, HTTP, DNS, VPNs, firewalls, IDS/IPS, EDR, or endpoint agents. It gives these systems a shared, authenticated record for deciding whether an object or action is currently allowed. [file:1]

Best final design: a CLSVH Security Order is a compact, cryptographically protected, time- and event-bounded authorization. It says what object is involved, who or what is acting, what action is allowed, where it is allowed, how long the permission lasts, what evidence supports it, and who issued it.

Connection tracking alone says only that packets belong to an active flow. CLSVH adds object lineage, subject identity, destination/port/protocol scope, current security state, hard expiry, replay protection, evidence reference, policy binding, and revocation.

TTL is only an analogy. TTL is a small routing field that routers decrement. CLSVH should be compact and lifecycle-oriented, but it must not modify TTL or use hop count as authorization. CLSVH validity changes through time, policy, evidence, revalidation, and revocation.

The security problem

Open listening ports are necessary for many services, but a port open on a firewall is not proof that every sender, device, process, file, or workload should be trusted. A user can be compromised, a session can be stolen, a valid device can later become infected, and a file initially considered safe can produce malicious child processes.

Zero Trust supports the central CLSVH goal: do not grant trust merely because a device is internal, known, or already connected. Protected-resource access should be based on current policy and security context. [web:33][web:106]

CLSVH changes the decision from “does the source match an IP/port rule?” to “does this device, user/service, process/workload, object, and session hold a current authentic authorization for this exact objective?”

Companion model

IP still routes packets. TCP and UDP still transport data. TLS or mutual TLS still encrypts data and authenticates peers. HTTP still carries requests. The firewall still enforces packet and connection policy. EDR, IDS/IPS, Zeek, Suricata, auditd, eBPF, and endpoint agents still collect evidence. CLSVH provides the shared security identity, validation status, objective, expiry, and revocation information that these systems can enforce.

For TLS, the distinction is important: TLS answers “is data confidential and protected in transit?” CLSVH answers “should this authenticated device, user, process, file lineage, or workload be permitted to use this encrypted connection for this purpose now?” TLS extension work is a possible future profile, but a proxy, sidecar, or session-binding deployment is safer for the first prototype. [web:151][web:154][web:156]

Object identity and lineage

Use SOID as the stable Security Object ID for one logical object or lineage node. Use CID as a cryptographic content ID, such as SHA-256, for exact bytes. Use OID for an individual observation/event. Use Parent SOID when a new object is derived from another object. Use instance identifiers for concrete objects such as a PID plus start time, inode plus mount namespace, or network flow/session.

Example: a downloaded document receives SOID-A. The completed file receives a CID. An application open creates an observation OID. If the document launches a script, create SOID-B with Parent SOID-A. If the script makes outbound TLS traffic, bind that flow to SOID-B. If the script becomes malicious, revoke the authorization, block egress, and retain the evidence chain.

W3C PROV is useful research material because it represents entities, activities, agents, derivation, generation, usage, and lifecycle relationships. Those concepts map well to CLSVH file, process, sensor, and lineage records. [web:162][web:166][web:170]

Security Order model

A CLSVH Security Order is the key operational idea. It is a digitally authenticated instruction from a trusted authority: a named or pseudonymous subject may perform one stated action against one target under defined security conditions.

Example order: "Managed device D, authenticated employee U, approved browser/process P, and SOID-N may connect using TCP/TLS only to hr-api.internal on port 443 until 10:05 PM, while device posture remains acceptable and no revocation or suspicious lineage event occurs."

Orders must be issued by a trusted validation authority, never self-declared by an untrusted sender. The order should be signed or MAC-protected, short-lived, bound to destination and action, and revocable.

Recommended core fields

Core fields: Version; Flags; Issuer ID; Key ID; Assertion or Order ID; SOID; Parent SOID; Object Class; Layer/Context; Subject Reference; Content Digest; Destination Service; Destination Port; Protocol; Objective/Allowed Action; Security State; Risk/Profile ID; Not-Before; Expiry; Invalidation Event Mask; Evidence Reference and Hash; Sequence or Nonce; Previous Binding Digest; and Authentication.

Core states: UNKNOWN, PENDING, VALIDATED, SUSPICIOUS, QUARANTINED, MALICIOUS, REVOKED, and EXPIRED. A verifier must understand all critical fields before relying on an assertion. Unknown critical extensions require safe fallback such as deny, pending, or revalidation. [file:1]

CBOR and COSE are strong format candidates. CBOR is designed for compact message size, and COSE supports protected headers, signatures, message authentication codes, and encryption. COSE_Sign1 is suitable when independent parties verify a signature; COSE_Mac0/HMAC can be suitable inside one controlled trust domain. [web:165][web:111]

Lifecycle and validation lifetime

The full lifecycle is: Ingest, Identify, Observe, Validate, Bind, Enforce, Revalidate, Revoke or Quarantine, and Close. Do not overwrite old security decisions. Append a new assertion with a new sequence and previous-binding digest so investigators can review changes over time. [file:1]

CLSVH Security Validation Lifetime must be more than a timestamp. An order remains valid only while the time window is valid, issuer authentication verifies, sequence is fresh, no revocation exists, relevant content digest still matches, required evidence is present, and no invalidating event has occurred.

Invalidating events include file modification, archive extraction, process-image change, suspicious child-process creation, memory injection, user/privilege change, destination/zone/protocol change, expired identity session, policy change, agent-health failure, replay detection, IDS/IPS alert, and explicit revocation.

Fast path and slow path

The design must not perform a full cryptographic signature verification, file hash verification, and evidence-database lookup for every packet. Use a slow path for new sessions and changed conditions; use a fast path for packets that already belong to an approved flow.

On the slow path, validate issuer, signature/MAC, identity, device/workload posture, SOID state, evidence, policy, destination, port, protocol, expiry, and replay controls. Then create short local firewall state such as a connection mark, session ID, SOID reference, authorization expiry, allowed scope, and revocation version.

On the fast path, the firewall checks only local state: whether the packet is in an approved flow, whether the hard authorization expiry has passed, whether revocation is present, and whether target scope still matches. If an event changes risk, the control plane revokes the flow; the firewall removes state and blocks later packets.

Linux nftables connection tracking exposes flow state and expiry, but ordinary active traffic can refresh normal connection timeouts. CLSVH therefore needs its own hard authorization expiry that cannot be extended merely by sending packets. [web:121][web:126]

Overhead and latency

The best default profile is “full Security Order at session setup plus firewall connection state afterward.” This produces a slow-path cost for new or renewed sessions but nearly no per-packet CLSVH overhead once a flow is approved.

Do not carry full certificates, full evidence, large hashes, or full signatures in every packet. Store evidence externally. If a private controlled overlay later needs per-packet assurance, use only a compact reference plus short MAC; treat this as advanced research, not the first deployment.

Use local cache and revocation data near the firewall. Perform asynchronous evidence logging when possible. Revalidate proactively before expiry. Use event-driven revalidation instead of querying every packet. eBPF/XDP can provide fast local packet drop or redirect decisions at the Linux network interface when later performance work justifies it. [web:109]

Firewall enforcement

Use default deny only for protected internal services, not for all public Internet traffic. A protected SSH, database, API, remote desktop, Kubernetes, file-share, or internal web service should accept access only through an enforcement point that has verified a valid Security Order.

Example enforcement: if order is missing, issuer is untrusted, cryptography fails, the order is expired, sequence is replayed, destination/port mismatch occurs, state is malicious/revoked, or required agent/device proof fails, deny or quarantine. If state is pending, permit only validation, update, identity, DNS/time, or remediation paths. If validated, permit only the precise action and scope.

For unknown external traffic, use silent drop or generic denial instead of detailed error messages. For managed internal devices, return status through a secure agent, application, or management channel. Do not use ARP to report or carry CLSVH decisions.

ARP, device trust, and hijacking

ARP is not an identity or security-authorization protocol. It maps IPv4 addresses to MAC addresses locally and can be spoofed. Use it only as defense in depth with switch port security, 802.1X, DHCP/IP-MAC binding, segmentation, and Dynamic ARP Inspection. DAI can discard ARP messages inconsistent with trusted bindings. [web:54]

Protect against hijacking by binding each order to device identity or mTLS session, user/service identity, process/workload where possible, SOID, destination, port, protocol, objective, policy, expiry, sequence/nonce, and cryptographic issuer. Use short lifetimes, replay cache, revocation, content digest checks, parent-child lineage, and gateway-only paths to protected services.

Do not fully trust the endpoint agent. Use least privilege, protected keys, code-signed updates, heartbeat checks, secure or measured boot when available, remote attestation, and independent gateway/IDS evidence. The RATS architecture separates attester, verifier, and relying party, which supports this design. [web:101][web:49]

Deployment options

Option 1: Sidecar/reference profile. Keep detailed evidence in a trusted metadata store and pass only SOID/assertion references. This is the recommended foundation.

Option 2: Firewall-session profile. Verify a Security Order once and install temporary connection/session state. This is the best firewall prototype because it has low steady-state packet overhead.

Option 3: File metadata and runtime-event profiles. Store a reference in a file xattr or sidecar and link audit/eBPF/EDR events to SOID. This supports file-to-process-to-network lineage.

Option 4: Proxy/API profile. A reverse proxy or API gateway checks CLSVH order per request or session. Good for internal web applications, APIs, uploads, and downloads.

Option 5: mTLS/workload profile. Bind Security Orders to verified device or workload sessions. Good for services, containers, and microservices.

Option 6: VPN/overlay or per-packet private label. Use only after other profiles work, in controlled enterprise environments. Do not rely on public Internet IP header modification because protocol extensions can have middlebox and operational compatibility problems. [web:108][web:136]

Organization uniqueness and standardization

A universal CLSVH standard should standardize semantic meaning, not force all organizations to use the same employee names, device IDs, policies, or keys. Standardize version, core fields, object classes, state meanings, action vocabulary, extension criticality, encoding, cryptographic profiles, lifetime/replay rules, reason codes, and conformance behavior.

Allow each organization to choose internal identity system, SOID namespace, risk scoring model, policy IDs, trust anchors, evidence store, retention rules, agent technology, firewall platform, and private non-critical extensions. Use opaque or pseudonymous references across trust boundaries.

For standards work, publish a research specification, canonical CBOR/COSE profile, deterministic test vectors, invalid-input tests, registry specification, and two independent implementations. Protocol-extension guidance emphasizes explicit extensibility and interoperability; IANA registry guidance requires clear registration and change policies. [web:136][web:138][web:145][web:141]

Prototype roadmap

Phase 1: Freeze CLSVH v0.1 terminology, object model, core header, state machine, Security Order, evidence model, threat model, privacy model, reason codes, expiry/revocation rules, and test vectors.

Phase 2: Build an HTTPS ingress gateway that creates SOID-A and records source/session evidence. Use SQLite or PostgreSQL for the evidence store. Compute SHA-256 CID for received files. Use a rule-based policy engine initially.

Phase 3: Build Linux endpoint evidence capture using auditd, eBPF, Falco, or selected LSM hooks. Monitor file create/write/open, exec, child process, network connect, privilege change, and carefully selected memory events.

Phase 4: Integrate nftables firewall session/connection marks. Validate order on new protected connection; use local expiry/revocation checks on established flows. Implement quarantine directory, process termination, and egress block.

Phase 5: Begin with HMAC/COSE_Mac0 in one lab trust domain. Add Ed25519/COSE_Sign1 for independent gateway/verifier or cross-organization research. Add mTLS/session binding later.

Phase 6: Build a second independent implementation, publish conformance tests, and measure security, performance, reliability, privacy, and interoperability.

Evaluation plan

Compare CID only, SOID only, SOID plus CID, and SOID plus CID plus Parent SOID for correlation precision/recall across file copying, editing, decoding, archive extraction, child processes, shared libraries, and process inheritance.

Compare normal stateful firewall, CLSVH session-level MAC, CLSVH session-level signature, and optional per-packet private-overlay MAC. Measure connection setup latency, p50/p95/p99 latency, packets/connections per second, CPU/RAM, state-table size, token size, cache hit rate, event loss, and revocation time.

Test missing order, invalid signature, unknown issuer, expired order, replayed sequence, wrong destination/port, file hash mismatch, suspicious child process, malicious outbound TLS attempt, agent failure, policy outage, evidence-store outage, and explicit revocation.

Measure detection rate, false positives, false negatives, time-to-decision, revalidation latency, quarantine accuracy, and time from risk event to blocked traffic. Do not claim fixed performance or security values until measured on the actual prototype.

Further research topics

Continuous and remote attestation; supply-chain and SBOM provenance; C2PA-style signed content provenance comparison; service mesh and SPIFFE/SPIRE workload identity; data-centric authorization; secure DNS/destination intent; privacy-preserving selective disclosure; formal verification of state and expiry rules; high availability; SmartNIC/DPU/XDP acceleration; adversarial parser and cache-poisoning tests; post-quantum algorithm agility; and AI-assisted triage with deterministic policy as the final enforcement authority.

Useful open-source research components include nftables/conntrack for firewall state, eBPF/XDP for fast Linux paths, Zeek for network analysis, Suricata for IDS/IPS, Falco for runtime security, auditd for Linux auditing, Open Policy Agent or a simple rules engine, PostgreSQL/SQLite for evidence, Redis for short-lived cache/revocation, and OpenSearch/Grafana for investigation. Suricata, Zeek, and Falco are relevant sources for network and runtime evidence collection. [web:71][web:73][web:82]

Final recommendation

Build CLSVH as a transport-neutral, objective-oriented Security Order and lineage framework. Start with sidecar evidence, firewall-session enforcement, Linux file/process monitoring, short hard authorization lifetimes, event-driven revalidation, revocation, and cryptographic protection. Keep public Internet packet formats unchanged. Add TLS/session binding, controlled overlays, and per-packet labels only as later profiles.

The final research claim should be: CLSVH provides a shared, cryptographically protected validation contract for existing security and networking systems. It identifies the object and its lineage; binds evidence and integrity; authorizes a precise objective; limits trust by time and security events; prevents tampering and replay; and allows existing firewalls, proxies, TLS gateways, endpoint controls, and workloads to make explainable, current, and revocable access decisions. [file:1]

Research sources used

User-provided CLSVH design document: object model, core fields, cryptographic binding, validation lifetime, end-to-end example, registries, standardization strategy, and prototype scope. [file:1]

NIST SP 800-207 and SP 800-207A: Zero Trust and policy-based protected-resource access. [web:33][web:106]

W3C PROV-DM and PROV-O: entity, activity, agent, derivation, and lifecycle concepts. [web:162][web:166][web:170]

RFC 9052 COSE/CBOR: compact cryptographic structures. [web:165][web:111]

RFC 9334 RATS and Entity Attestation Token materials: device/agent trust roles. [web:101][web:49]

RFC 6709 and RFC 9413: extension and robust protocol design. [web:136][web:154]

RFC 8126/IANA guidance: registries and protocol parameters. [web:138][web:145]

RFC 9098: IPv6 extension-header operational considerations. [web:108]

nftables/conntrack documentation: stateful tracking and expiration. [web:121][web:126]

Suricata, Zeek, and Falco materials. [web:71][web:73][web:82]